3841. Palindromic Path Queries in a Tree

Daily Question - October 29, 2026

Problem Statement

You are given an undirected tree with n nodes labeled 0 to n - 1, represented by edges, and a lowercase string s of length n. Character s[i] is assigned to node i.

Process the string array queries in order. Each query is either:

update ui c: Change the character at node ui to c.

query ui vi: Determine whether the characters on the unique path from ui to vi, including both endpoints, can be rearranged into a palindrome.

Return a List<Boolean> containing one result for each query operation, in order. Updates do not produce an output entry.

Examples

Example 1: n = 3, edges = [[0,1],[1,2]], s = "aac", queries = ["query 0 2", "update 1 b", "query 0 2"]. Output: [true, false]. The first path gives "aac", which can form "aca". After the update, the path gives "abc", which cannot form a palindrome.

Example 2: n = 4, edges = [[0,1],[0,2],[0,3]], s = "abca", queries = ["query 1 2", "update 0 b", "query 2 3", "update 3 a", "query 1 3"]. Output: [false, false, true]. The queried path strings are "bac", "cba", and "bba"; only the last can form a palindrome.

Constraints

1 <= n == s.length <= 50,000; edges.length == n - 1. The edges form a valid tree. Node indices are in [0, n - 1]. All letters are lowercase English letters. There are between 1 and 50,000 queries, each in one of the formats above.

Approach

1. Character parity: Use a 26-bit integer mask. Bit c is set when character c occurs an odd number of times. A collection of characters can form a palindrome exactly when its mask has at most one set bit: (mask & (mask - 1)) == 0.

2. Root-path masks: Root the tree at node 0. Let P(u) be the XOR of character bits on the root-to-u path, including u. If w = LCA(u, v), then the path mask is P(u) XOR P(v) XOR bit(s[w]). The common prefix cancels, so the LCA character must be restored once.

3. Updates: Changing a node character changes P(x) for every node x in its subtree. An iterative DFS numbers nodes so each subtree is one interval [tin[u], tin[u] + subtreeSize[u] - 1].

4. Fenwick tree: Store only changes to the original root-path masks. For delta = oldBit XOR newBit, XOR delta at the interval start and immediately after its end. A Fenwick prefix query gives the accumulated change at a node. Thus current P(u) = initialPrefix[u] XOR fenwick.get(tin[u]).

5. LCA: Binary lifting finds the lowest common ancestor in O(log n). The DFS is iterative to avoid recursion depth issues on long chains.

Complete Java Solution

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

class Solution {
    private int[][] up;
    private int[] depth;
    private int log;

    public List<Boolean> palindromePath(
            int n, int[][] edges, String s, String[] queries) {
        int[] head = new int[n];
        Arrays.fill(head, -1);
        int[] to = new int[2 * edges.length];
        int[] next = new int[2 * edges.length];
        int edgeIndex = 0;

        for (int[] edge : edges) {
            int u = edge[0];
            int v = edge[1];
            to[edgeIndex] = v;
            next[edgeIndex] = head[u];
            head[u] = edgeIndex++;
            to[edgeIndex] = u;
            next[edgeIndex] = head[v];
            head[v] = edgeIndex++;
        }

        log = 1;
        while ((1 << log) <= n) {
            log++;
        }
        up = new int[log][n];
        depth = new int[n];
        int[] tin = new int[n];
        int[] subtreeSize = new int[n];
        int[] prefix = new int[n];
        int[] order = new int[n];
        int[] stack = new int[n];
        char[] letters = s.toCharArray();
        int stackSize = 0;
        int timer = 0;
        stack[stackSize++] = 0;
        up[0][0] = 0;
        prefix[0] = 1 << (letters[0] - 'a');

        while (stackSize > 0) {
            int u = stack[--stackSize];
            tin[u] = ++timer;
            order[timer - 1] = u;
            subtreeSize[u] = 1;
            for (int e = head[u]; e != -1; e = next[e]) {
                int v = to[e];
                if (v == up[0][u]) {
                    continue;
                }
                up[0][v] = u;
                depth[v] = depth[u] + 1;
                prefix[v] = prefix[u] ^ (1 << (letters[v] - 'a'));
                stack[stackSize++] = v;
            }
        }

        for (int index = n - 1; index > 0; index--) {
            int u = order[index];
            subtreeSize[up[0][u]] += subtreeSize[u];
        }
```

```
        for (int level = 1; level < log; level++) {
            for (int u = 0; u < n; u++) {
                up[level][u] = up[level - 1][up[level - 1][u]];
            }
        }

        Fenwick fenwick = new Fenwick(n);
        List<Boolean> answer = new ArrayList<>();
        for (String query : queries) {
            String[] parts = query.split(" ");
            int u = Integer.parseInt(parts[1]);
            if (parts[0].equals("update")) {
                char newCharacter = parts[2].charAt(0);
                int delta = (1 << (letters[u] - 'a'))
                        ^ (1 << (newCharacter - 'a'));
                int left = tin[u];
                int right = tin[u] + subtreeSize[u] - 1;
                fenwick.add(left, delta);
                fenwick.add(right + 1, delta);
                letters[u] = newCharacter;
            } else {
                int v = Integer.parseInt(parts[2]);
                int ancestor = lca(u, v);
                int maskU = prefix[u] ^ fenwick.get(tin[u]);
                int maskV = prefix[v] ^ fenwick.get(tin[v]);
                int pathMask = maskU ^ maskV
                        ^ (1 << (letters[ancestor] - 'a'));
                answer.add((pathMask & (pathMask - 1)) == 0);
            }
        }
        return answer;
    }

    private int lca(int u, int v) {
        if (depth[u] < depth[v]) {
            int temporary = u;
            u = v;
            v = temporary;
        }
        int difference = depth[u] - depth[v];
        for (int level = 0; level < log; level++) {
            if ((difference & (1 << level)) != 0) {
                u = up[level][u];
            }
        }
        if (u == v) {
            return u;
        }
        for (int level = log - 1; level >= 0; level--) {
            if (up[level][u] != up[level][v]) {
                u = up[level][u];
                v = up[level][v];
            }
        }
        return up[0][u];
    }

    private static class Fenwick {
        private final int[] tree;
        Fenwick(int n) {
            tree = new int[n + 1];
        }
        void add(int index, int value) {
            while (index < tree.length) {
                tree[index] ^= value;
                index += index & -index;
            }
        }
        int get(int index) {
            int result = 0;
            while (index > 0) {
                result ^= tree[index];
                index -= index & -index;
            }
```

```
            return result;
        }
    }
}
```

Correctness

The initial prefix masks represent exactly the root-to-node paths. Each update toggles exactly the two affected letter bits on every root path containing the updated node, which is exactly its subtree. The Fenwick interval update therefore preserves correct current prefix masks. XORing the two current prefixes and restoring the LCA yields exactly the parity mask of the requested path. The at-most-one-set-bit test is precisely the palindrome rearrangement condition, so every returned answer is correct.

Complexity

For q queries: preprocessing O(n log n); each update O(log n); each path query O(log n). Total O((n + q) log n). Auxiliary space O(n log n), plus O(q) in the worst case for the returned results.

Implementation Notes

Use the exact signature List<Boolean> palindromePath(int n, int[][] edges, String s, String[] queries). A query from a node to itself always returns true. Updating a node to the same letter produces delta zero. The code supports n = 1 and updates to the root. Copy only Java code into the editor, not Markdown backticks or links.